

CoreAVR + ArduinoKit.

Swift on the Arduino, the Swift way.



Who am I?

Brent Van den Abbeel

Who am I?

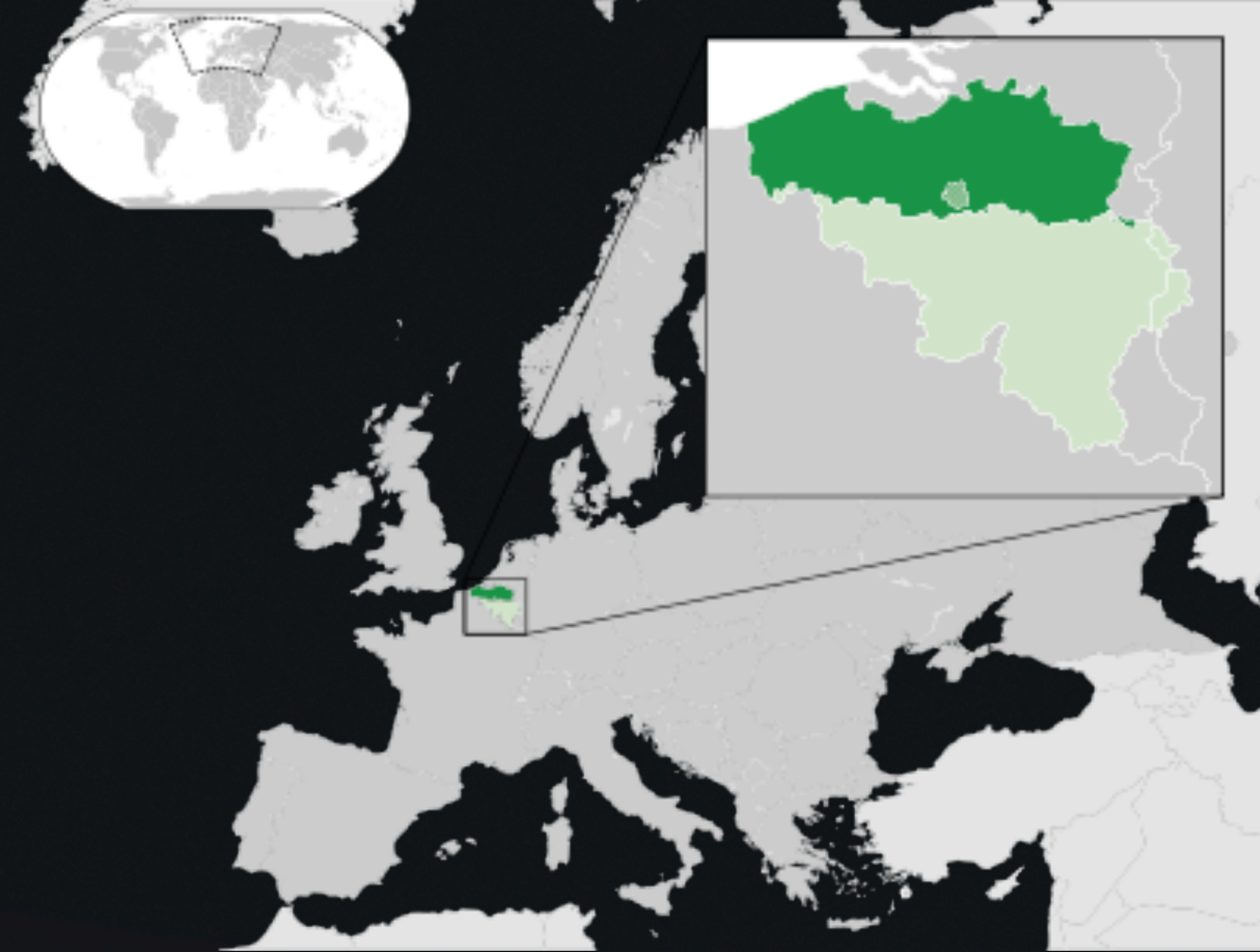
Brent Van den Abbeel

- 21 years old

Who am I?

Brent Van den Abbeel

- 21 years old
- Flanders, Belgium



Who am I?

Brent Van den Abbeel

- 21 years old
- Flanders, Belgium
- Software Developer (Swift + JVM)

Who am I?

Brent Van den Abbeel

- 21 years old
- Flanders, Belgium
- Software Developer (Swift + JVM)
- Somewhat into rockets



What am I here for?

Swift for Arduino.



What am I here for?

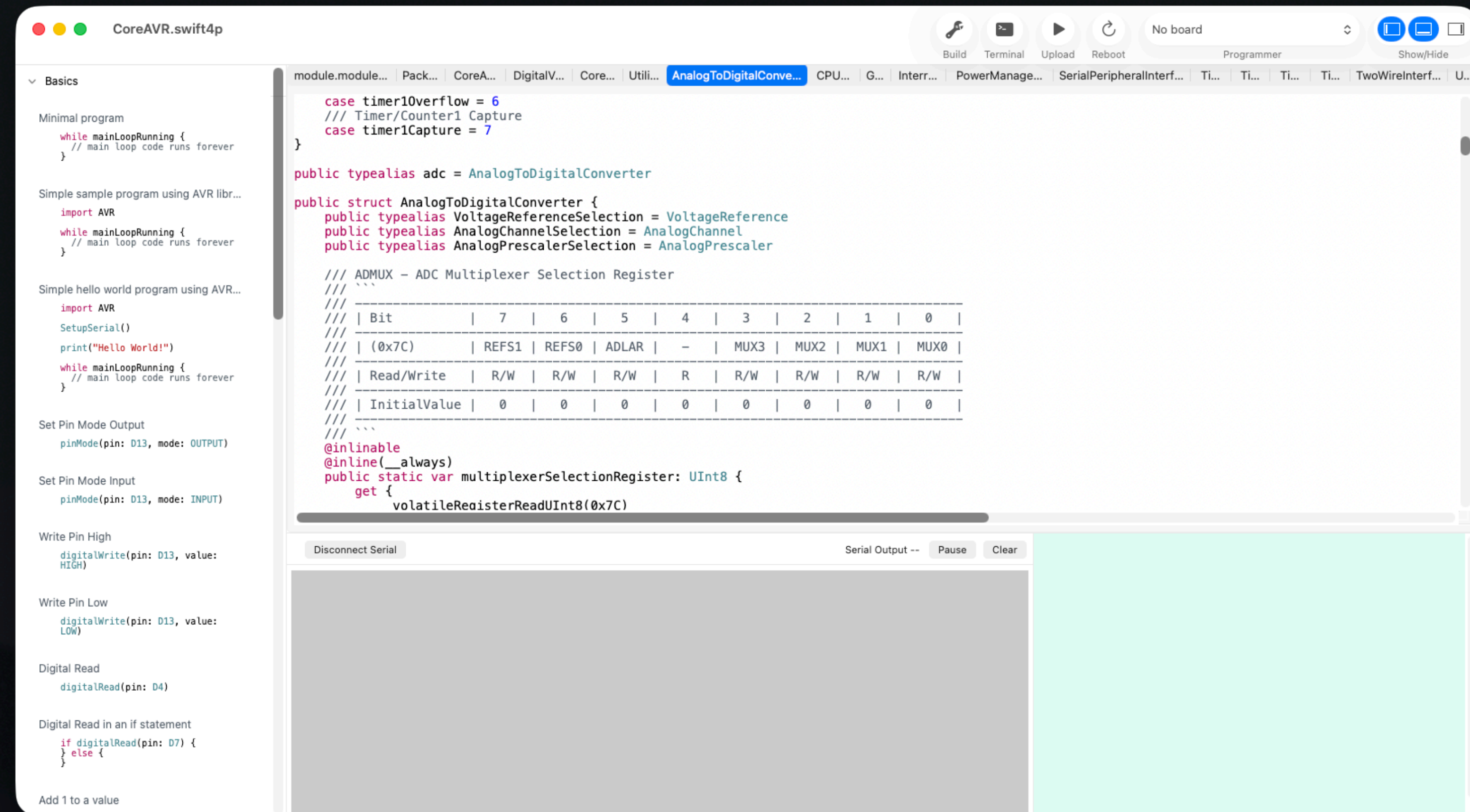
Swift for Arduino.

- Goal: Running Swift on the Arduino

What am I here for?

Swift for Arduino.

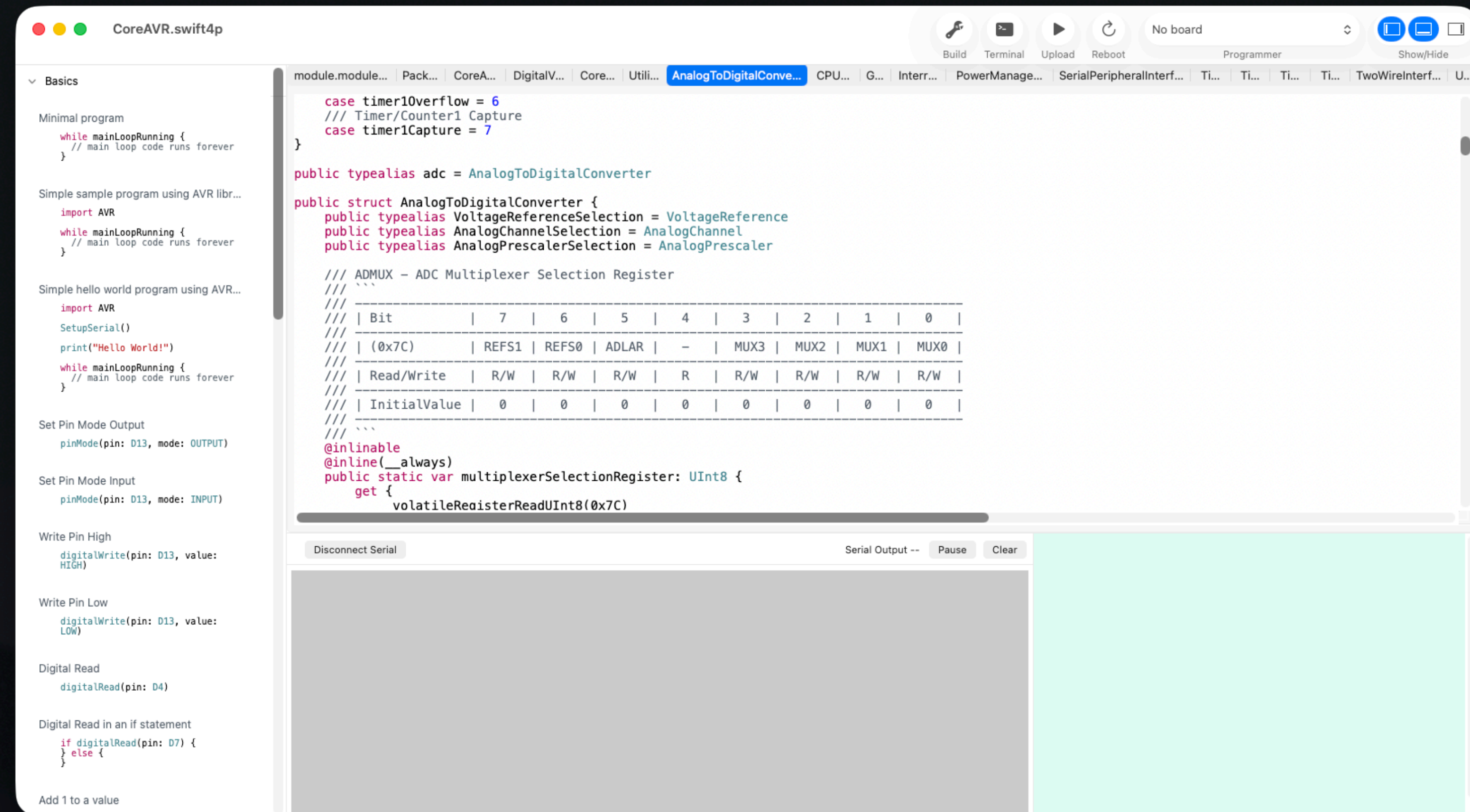
- Goal: Running Swift on the Arduino
- How: S4A IDE + μ Swift Toolchain



What am I here for?

Swift for Arduino.

- Goal: Running Swift on the Arduino
- How: S4A IDE + μ Swift Toolchain
- Actively being developed



About CoreAVR.

A Swift-first AVR HAL.



About CoreAVR.

A Swift-first AVR HAL.

- Hardware Abstraction Layer for AVR chips

About CoreAVR.

A Swift-first AVR HAL.

- Hardware Abstraction Layer for AVR chips
- Fully written in Swift*

About CoreAVR.

A Swift-first AVR HAL.

- Hardware Abstraction Layer for AVR chips
- Fully written in Swift*
- Adaptable to other chips through HALGEN

Going in-depth.

CoreAVR



Going in-depth.

CoreAVR

```
#include <avr/io.h>

int main(void) {
    DDRB |= (1 << PB3);

    TCCR2A = (1 << WGM21) | (1 << WGM20);
    TCCR2B = (1 << WGM22);
    TCCR2A |= (1 << COM2A1);
    TCCR2B |= (1 << CS22);
    OCR2A = 128;

    while (1) {}
}
```


Going in-depth.

CoreAVR

```
#include <avr/io.h>
```

```
int main(void) {  
    DDRB |= (1 << PB3);  
  
    TCCR2A = (1 << WGM21) | (1 << WGM20);  
    TCCR2B = (1 << WGM22);  
    TCCR2A |= (1 << COM2A1);  
    TCCR2B |= (1 << CS22);  
    OCR2A = 128;  
  
    while (1) {}  
}
```

```
import CoreAVR
```

```
// Setup for Analog Write on PB3  
// pin 11 on the Arduino Uno  
GPIO.pb3.setDataDirection(.output)  
  
timer2.compareOutputModeA = .clear  
timer2.waveformGenerationMode = .fastPWM  
timer2.prescaler = .running64  
timer2.outputCompareRegisterA = 128
```


Going in-depth.

CoreAVR

```
#include <avr/io.h>

int main(void) {
    DDRB |= (1 << PB3); // OC2A - Arduino Pin 11 (PORTB bit 3)

    TCCR2A = (1 << WGM21) | (1 << WGM20); // Fast PWM
    TCCR2B = (1 << WGM22);
    TCCR2A |= (1 << COM2A1); // Clear OC2A on compare match
    TCCR2B |= (1 << CS22); // Set prescaler to /64
    OCR2A = 128; // Duty cycle 50%

    while (1) {}
}
```

```
import CoreAVR
```

```
// Setup for Analog Write on PB3
// pin 11 on the Arduino Uno
GPIO.pb3.setDataDirection(.output)

timer2.compareOutputModeA = .clear
timer2.waveformGenerationMode = .fastPWM
timer2.prescaler = .running64
timer2.outputCompareRegisterA = 128
```


Going in-depth.

CoreAVR

```
#include <avr/io.h>

int main(void) {
    DDRB |= (1 << PB3); // OC2A - Arduino Pin 11 (PORTB bit 3)

    TCCR2A = (1 << WGM21) | (1 << WGM20); // Fast PWM
    TCCR2B = (1 << WGM22);
    TCCR2A |= (1 << COM2A1); // Clear OC2A on compare match
    TCCR2B |= (1 << CS22); // Set prescaler to /64
    OCR2A = 128; // Duty cycle 50%

    while (1) {}
}
```

```
import CoreAVR
```

```
// Setup for Analog Write on PB3
// pin 11 on the Arduino Uno
GPIO.pb3.setDataDirection(.output)

timer2.compareOutputModeA = .clear
timer2.waveformGenerationMode = .fastPWM
timer2.prescaler = .running64
timer2.outputCompareRegisterA = 128
```

Sketch uses 174 bytes (0%) of program storage space. Maximum is 32256 bytes.
Global variables use 0 bytes (0%) of dynamic memory, leaving 2048 bytes for local variables. Maximum is 2048 bytes.

Going in-depth.

CoreAVR

```
#include <avr/io.h>

int main(void) {
    DDRB |= (1 << PB3); // OC2A - Arduino Pin 11 (PORTB bit 3)

    TCCR2A = (1 << WGM21) | (1 << WGM20); // Fast PWM
    TCCR2B = (1 << WGM22);
    TCCR2A |= (1 << COM2A1); // Clear OC2A on compare match
    TCCR2B |= (1 << CS22); // Set prescaler to /64
    OCR2A = 128; // Duty cycle 50%

    while (1) {}
}
```

```
import CoreAVR
```

```
// Setup for Analog Write on PB3
// pin 11 on the Arduino Uno
GPIO.pb3.setDataDirection(.output)

timer2.compareOutputModeA = .clear
timer2.waveformGenerationMode = .fastPWM
timer2.prescaler = .running64
timer2.outputCompareRegisterA = 128
```

```
Sketch uses 174 bytes (0%) of program storage space. Maximum is 32256 bytes.
Global variables use 0 bytes (0%) of dynamic memory, leaving 2048 bytes for local variables. Maximum is 2048 bytes.
```

Linked ELF

```
code: 0K (180 bytes)
global vars: 0K (0 bytes)
```

```
***
Built successfully
```


Going (a bit more) in-depth.

CoreAVR

Going (a bit more) in-depth.

CoreAVR

```
#include <avr/io.h>
#include <util/setbaud.h>

void uart_init(void) {
    UBBR0H = UBBRH_VALUE;
    UBBR0L = UBBRL_VALUE;
    UCSR0B = (1 << TXEN0);
    UCSR0C = (1 << UCSZ01) | (1 <<
UCSZ00);
}

void uart_putchar(char c) {
    while (!(UCSR0A & (1 << UDRE0)));
    UDR0 = c;
}

void uart_puts(const char *str) {
    while (*str) {
        uart_putchar(*str++);
    }
}

int main(void) {
    uart_init();
    uart_puts("Hello, World!\n");
}
```


Going (a bit more) in-depth.

CoreAVR

```
#include <avr/io.h>
#include <util/setbaud.h>

void uart_init(void) {
    UBBR0H = UBBRH_VALUE;
    UBBR0L = UBBRL_VALUE;
    UCSR0B = (1 << TXEN0);
    UCSR0C = (1 << UCSZ01) | (1 <<
UCSZ00);
}

void uart_putchar(char c) {
    while (!(UCSR0A & (1 << UDRE0)));
    UDR0 = c;
}

void uart_puts(const char *str) {
    while (*str) {
        uart_putchar(*str++);
    }
}

int main(void) {
    uart_init();
    uart_puts("Hello, World!\n");
}
```

```
import CoreAVR

uart0.baudRate = 57600
uart0.transmitterEnable = .on

uart0.write("Hello, World!\n")
```


Going (a bit more) in-depth.

CoreAVR

```
#include <avr/io.h>
#include <util/setbaud.h>

void uart_init(void) {
    UBBR0H = UBBRH_VALUE;
    UBBR0L = UBBRL_VALUE;
    UCSR0B = (1 << TXEN0);
    UCSR0C = (1 << UCSZ01) | (1 <<
UCSZ00);
}

void uart_putchar(char c) {
    while (!(UCSR0A & (1 << UDRE0)));
    UDR0 = c;
}

void uart_puts(const char *str) {
    while (*str) {
        uart_putchar(*str++);
    }
}

int main(void) {
    uart_init();
    uart_puts("Hello, World!\n");
}
```

```
import CoreAVR
```

```
uart0.baudRate = 57600
uart0.transmitterEnable = .on

uart0.write("Hello, World!\n")
```

Sketch uses 222 bytes (0%) of program storage space. Maximum is 32256 bytes.
Global variables use 16 bytes (0%) of dynamic memory, leaving 2032 bytes for local variables. Maximum is 2048 bytes.

Going (a bit more) in-depth.

CoreAVR

```
#include <avr/io.h>
#include <util/setbaud.h>

void uart_init(void) {
    UBBR0H = UBBRH_VALUE;
    UBBR0L = UBBRL_VALUE;
    UCSR0B = (1 << TXEN0);
    UCSR0C = (1 << UCSZ01) | (1 <<
UCSZ00);
}

void uart_putchar(char c) {
    while (!(UCSR0A & (1 << UDRE0)));
    UDR0 = c;
}

void uart_puts(const char *str) {
    while (*str) {
        uart_putchar(*str++);
    }
}

int main(void) {
    uart_init();
    uart_puts("Hello, World!\n");
}
```

```
import CoreAVR

uart0.baudRate = 57600
uart0.transmitterEnable = .on

uart0.write("Hello, World!\n")
```

```
Linked ELF

code: 0K (534 bytes)
global vars: 0K (16 bytes)

***
Built successfully
```

Sketch uses 222 bytes (0%) of program storage space. Maximum is 32256 bytes.
Global variables use 16 bytes (0%) of dynamic memory, leaving 2032 bytes for local variables. Maximum is 2048 bytes.

Transparency.

CoreAVR



Transparency.

CoreAVR

- Fully open-source (CoreAVR + HALGEN) (MIT)
- CoreAVR: <https://github.com/pdshelley/CoreAVR>
- HALGEN: <https://github.com/pdshelley/HALGEN>

About ArduinoKit.

The S4A Arduino Standard Library.



About ArduinoKit.

The S4A Arduino Standard Library.

- Standard Arduino Library for S4A



About ArduinoKit.

The S4A Arduino Standard Library.

- Standard Arduino Library for S4A
- Built on (and fully exposing) CoreAVR



About ArduinoKit.

The S4A Arduino Standard Library.

- Standard Arduino Library for S4A
- Built on (and fully exposing) CoreAVR
- Fully written in Swift*



About ArduinoKit.

The S4A Arduino Standard Library.

- Standard Arduino Library for S4A
- Built on (and fully exposing) CoreAVR
- Fully written in Swift*
- “Dual” Architecture, allowing both Arduino-style programs and Swift programs



Going in-depth.

ArduinoKit



Going in-depth.

ArduinoKit

```
// the setup function runs once when you press reset or power the board
void setup() {
  pinMode(LED_BUILTIN, OUTPUT);
}

// the loop function runs over and over again forever
void loop() {
  digitalWrite(LED_BUILTIN, HIGH);
  delay(1000);
  digitalWrite(LED_BUILTIN, LOW);
  delay(1000);
}
```


Going in-depth.

ArduinoKit

```
// the setup function runs once when you press reset or power the board
void setup() {
    pinMode(LED_BUILTIN, OUTPUT);
}
```

```
// the loop function runs over and over again forever
void loop() {
    digitalWrite(LED_BUILTIN, HIGH);
    delay(1000);
    digitalWrite(LED_BUILTIN, LOW);
    delay(1000);
}
```

```
import ArduinoKit
```

```
// 0. Set up ArduinoKit
initializeArduino()
```

```
// 1. Setup
pinMode(pin: builtinLed, mode: .output)
```

```
// 2. Loop
while true {
    digitalWrite(pin: builtinLed, value: .high)
    delay(millis: 1000)
    digitalWrite(pin: builtinLed, value: .low)
    delay(millis: 1000)
}
```


A “Dual” Architecture.

A “Dual” Architecture.

ArduinoKit “default”

```
import ArduinoKit

// 0. Set up ArduinoKit
initializeArduino()

// 1. Setup
pinMode(pin: builtinLed, mode: .output)

// 2. Loop
while true {
    digitalWrite(pin: builtinLed, value: .high)
    delay(millis: 1000)
    digitalWrite(pin: builtinLed, value: .low)
    delay(millis: 1000)
}
```


A “Dual” Architecture.

ArduinoKit “default”

```
import ArduinoKit

// 0. Set up ArduinoKit
initializeArduino()

// 1. Setup
pinMode(pin: builtinLed, mode: .output)

// 2. Loop
while true {
    digitalWrite(pin: builtinLed, value: .high)
    delay(millis: 1000)
    digitalWrite(pin: builtinLed, value: .low)
    delay(millis: 1000)
}
```

ArduinoKit “classic”

```
import ArduinoKit

// 0. Set up ArduinoKit
initializeArduino()

// 1. Setup
pinMode(LED_BUILTIN, OUTPUT)

// 2. Loop
while true {
    digitalWrite(LED_BUILTIN, HIGH)
    delay(1000)
    digitalWrite(LED_BUILTIN, LOW)
    delay(1000)
}
```


A “Dual” Architecture.

ArduinoKit “default”

```
import ArduinoKit

// 0. Set up ArduinoKit
initializeArduino()

// 1. Setup
pinMode(pin: builtinLed, mode: .output)

// 2. Loop
while true {
    digitalWrite(pin: builtinLed, value: .high)
    delay(millis: 1000)
    digitalWrite(pin: builtinLed, value: .low)
    delay(millis: 1000)
}
```

Arduino Core

```
// 1. Setup
void setup() {
    pinMode(LED_BUILTIN, OUTPUT);
}

// 2. Loop
void loop() {
    digitalWrite(LED_BUILTIN, HIGH);
    delay(1000);
    digitalWrite(LED_BUILTIN, LOW);
    delay(1000);
}
```


Transparency.

ArduinoKit



Transparency.

ArduinoKit

- Fully open-source (MIT)
- <https://github.com/TheAlgorithm476/ArduinoKit>

About this blinking LED...

About this blinking LED...

```
import ArduinoKit

// 0. Set up ArduinoKit
initializeArduino()

// 1. Setup
pinMode(pin: builtinLed, mode: .output) // Sets up the builtin LED as an output.

// 2. Loop
while true {
    digitalWrite(pin: builtinLed, value: .high) // Turn on the LED (writes a "high" voltage)
    delay(millis: 1000) // Wait 1 second (1000 millis)
    digitalWrite(pin: builtinLed, value: .low) // Turn off the LED (writes a "low" voltage)
    delay(millis: 1000) // Wait 1 second (1000 millis)
}
```


Thanks for tuning in.

Feel free to reach out afterwards if you have any questions.
(I promise I won't bite!)

Swift for Arduino

<https://swiftforarduino.com>

swiftforarduino@gmail.com (commercial support)

CoreAVR: <https://github.com/pdshelley/CoreAVR>

HALGEN: <https://github.com/pdshelley/HALGEN>

ArduinoKit: <https://github.com/TheAlgorithm476/ArduinoKit>

Brent Van den Abbeel

<https://thealgorithm476.me>

GitHub, X, Discord: @thealgorithm476